IT 360 Cloud Integration | Week 3 | Team Lab 1 (Canvas 3.2, 40 points)

# Lab 1: Kubernetes on GKE with Helm and a NetworkPolicy

*Silver Lake Credit Union: run the loan calculator on Google Kubernetes Engine, roll it back with one command, and prove that only the front end can reach it*

---

**Version 2, November 2026.** Every command in this guide was run end to end on November 8, 2026, and the "You should see" boxes show the real output of that run. Your pod names, IP addresses and times will differ; the shape of the output will not.

> **Duration: about 80 minutes in total (75 in class plus 5 to get ready)**
>
> **Before class:** 5 minutes (start the Vocareum lab, open Cloud Shell).
>
> **In class:** 75 minutes of lab work in six parts (A 15, B 10, C 15, D 15, E 10, F 10). Part A is mostly waiting for Google to build the cluster; read Part B while it runs.
>
> **Instructor's timed run (November 8, 2026):** all four build parts, cluster included, took under 8 minutes of machine time; the cluster was RUNNING in under 5 minutes, the replacement pod appeared in 2 to 8 seconds, and the optional public IP arrived in 44 seconds.
>
> **Allow extra time if:** Google reports no capacity in your region (recreating the cluster adds 10 to 25 minutes) or Autopilot is adding a machine for the test pods (up to 90 seconds each).

## The story

Silver Lake Credit Union is moving its public loan calculator onto Google Kubernetes Engine (GKE). Last year an auditor wrote: "public-facing workloads must not be able to reach internal workloads on the same network." The infrastructure manager wants three things: the calculator running on Kubernetes, a release that can be rolled back with one command, and **proof** that pods without permission cannot reach it. A blocked connection is the proof.

## Lab objectives

By the end of this lab your team will be able to:

1. **Create a managed Kubernetes cluster** on Google Cloud (GKE Autopilot) and connect the kubectl command line to it.
2. **Deploy an application declaratively** with a YAML file, and explain how Kubernetes keeps the number of running copies you asked for (self-healing).
3. **Package and release the application with Helm**, upgrade it, and roll it back to an earlier version with one command, using Helm's revision history as an audit trail.
4. **Isolate the application with a Kubernetes NetworkPolicy** so only approved pods can reach it, and **prove** the control works with one blocked and one allowed connection.
5. **Test every network path** to the application (pod to pod, by DNS name, and from the internet through a cloud load balancer) and explain who can reach what.
6. **Collect evidence and clean up** cloud resources so nothing keeps billing.

### Why this lab matters for cloud integration

Kubernetes is the layer that makes clouds interchangeable: the same YAML and the same Helm chart you use today run unchanged on Google GKE, Amazon EKS, Azure AKS and DigitalOcean DOKS. Learning it once lets a team move or spread an application across providers.

Helm turns an application into a versioned package, so the same release can be installed in a test cluster on one cloud and a production cluster on another, and rolled back the same way everywhere.

A NetworkPolicy is security that travels with the application. It is written in Kubernetes terms (labels), not in one cloud's firewall language, so the isolation the auditor asked for holds wherever the cluster runs.

In Part E a Kubernetes Service of type LoadBalancer makes Google Cloud build a real load balancer for you. That is cloud integration in action: a Kubernetes object driving the cloud provider's own network services.

## The technologies in this lab

| Technology | What it is | Its job in this lab | Why it matters for cloud integration |
|---|---|---|---|
| **Kubernetes** | An open-source system that runs containers on a group of machines and keeps them in the state you describe | Runs the loan calculator: 3 copies, replaced automatically if one fails | The same API and YAML on every major cloud and on-premises, so workloads are portable |
| **GKE Autopilot** | Google's managed Kubernetes. Google runs the control plane and adds or removes machines (nodes) as your pods need them | Gives you a production-grade cluster in minutes without sizing servers | Each cloud offers managed Kubernetes (GKE, EKS, AKS, DOKS); you choose the provider, the skills stay the same |
| **kubectl** | The command-line client for the Kubernetes API | Applies YAML, lists pods, runs the test pods | One tool for every Kubernetes cluster, whichever cloud hosts it |
| **Deployment, ReplicaSet, Pod** | A Deployment says "keep N copies of this container"; it creates a ReplicaSet, which creates and replaces the Pods | Part B: three NGINX pods and the self-healing test | Desired-state automation: the cluster repairs itself instead of a person |
| **Service** | A stable name and virtual IP in front of a changing group of pods | Lets the test pods reach the app by name (loan-calc-loan-calc-chart) | Services of type LoadBalancer ask the cloud to create a real load balancer |
| **Helm** | The package manager for Kubernetes. A chart is a set of templates plus settings (values.yaml); an installed chart is a release with numbered revisions | Part C: install, upgrade and roll back the app | Repeatable, versioned releases across many clusters and clouds; one-command rollback |
| **NetworkPolicy** | A Kubernetes rule that says which pods may talk to which, chosen by labels. GKE enforces it with Dataplane V2 (eBPF on every node) | Part D: only role=frontend pods may reach the calculator | Zero-trust isolation defined with the app, portable across clouds, and testable |
| **Cloud Shell** | A Linux terminal in the browser, already signed in to your project, with gcloud, kubectl and helm installed | Where you run every command | No local setup: the same environment for every student, on Mac or Windows |

## How the parts fit together

1. **Part A** builds the platform: a Kubernetes cluster on Google Cloud, and a connection from Cloud Shell to it.
2. **Part B** runs the application the plain Kubernetes way and shows the cluster repairing itself.
3. **Part C** runs the same application as a Helm release, the way teams ship real software: versioned, upgradable, reversible.
4. **Part D** adds the security control the auditor asked for and proves it with a blocked and an allowed request.
5. **Part E** tests every network path, including one that crosses from the internet through a Google Cloud load balancer into the cluster.
6. **Part F** hands in the evidence and removes every billable resource.

## What you will build

| Part | What you do | Time |
|---|---|---|
| A. Cluster | Create a GKE Autopilot cluster named silverlake-public | 15 min (mostly waiting) |
| B. kubectl | Run 3 copies of the web server and watch Kubernetes heal itself | 10 min |
| C. Helm | Install the app as a Helm release, upgrade it, roll it back | 15 min |
| D. NetworkPolicy | Allow only role=frontend pods in, and prove it with two tests | 15 min |
| E. Connectivity | Test every network path, including from the internet (optional) | 10 min |
| F. Submit and clean up | Screenshots to Canvas, then delete the cluster | 10 min |
| Total | Six parts in one lab session | about 75 min, plus 5 min before class |

## Team roles

| Role | Owns |
|---|---|
| Cluster engineer | Part A: creates the cluster and connects kubectl |
| Deployment engineer | Part B: applies the Deployment and runs the self-healing check |
| Release engineer | Part C: the Helm install, upgrade and rollback |
| Security engineer | Parts D and E: the policy, the two probes, the screenshots for the auditor |

Everyone types along in their own Cloud Shell if they can, but the role owner drives that part on the shared screen.

## Before you start (5 minutes)

☐ Open Canvas and start the **Vocareum Google Cloud lab**, then click through to the Google Cloud console. (Using your own Google Cloud project instead? That works too; you need billing turned on.)

☐ At the top of the console, check the project selector shows your lab project.

☐ Open **Cloud Shell**: the >_ icon at the top right of the console. A terminal opens at the bottom of the page. Everything you type in this lab goes there.

☐ Cloud Shell already has gcloud, kubectl and helm. You install nothing, and it works the same on a Mac or Windows laptop because it runs in your browser.

☐ Bookmark the class repository: **https://github.com/isijackson/IT360-Fall-Labs-v2**. The lab files are in Week 3 labs/gke-helm-networkpolicy (the guide downloads the two YAML files from there for you in Parts B and D).

☐ Fill in the table below with your values.

| Item | Value in this guide | Yours |
|---|---|---|
| Project ID | jackson-isiko | |
| Region | us-central1 (if it is full, use us-east1) | |
| Cluster | silverlake-public | keep this name |
| App names | loan-calc (do not change) | keep these names |

## Where you run things

| Place | How to open it | What you do there |
|---|---|---|
| **Cloud Shell** (a Linux terminal inside your browser) | Google Cloud console, the >_ icon at the top right. It opens in your **home folder**; the prompt then ends in cloudshell:~ (your-project)$. If it shows cloudshell:/ or any other folder, type cd ~ to return home. | **Every command in this guide**: gcloud, kubectl, helm, curl, sed. Paste one block at a time and press Enter. |
| **Google Cloud console** (web pages) | https://console.cloud.google.com, the same browser tab | The **Click** steps: Kubernetes Engine, Clusters, Workloads, Services |
| **Your laptop's web browser** | Any tab | Part E only: open the public EXTERNAL-IP address |
| **Your laptop's own Terminal or PowerShell** | Not needed | Nothing. Do not run these commands on your laptop; Cloud Shell already has every tool and is signed in to your project. |

Every command block below starts with an orange ▸ **Run in:** line that tells you exactly where to run it.

> **How to read this guide**
>
> **Click:** numbered console steps, with a screenshot where it helps. The red numbers in the screenshot match the step numbers.
>
> **Type:** the same thing as commands for Cloud Shell. Copy one block at a time and press Enter.
>
> **You should see:** green boxes with the real output from the November 8 run.
>
> **If not:** yellow boxes with the fix for the problems students hit most often.
>
> Replace anything in angle brackets, like <PROJECT_ID>, including the brackets.

# Part A. Create the cluster (Cluster engineer, 15 min)

**Goal:** a running Kubernetes cluster, and kubectl connected to it. Autopilot means Google manages the machines for you; it also enforces NetworkPolicy by default, which Part D needs.

> **What this part does, and why**
>
> A cluster is the pool of computing power Kubernetes places your containers on. It has two halves: the **control plane** (the API server you talk to, the scheduler that decides where pods run, and the database that stores the desired state) and the **nodes** (the machines that run the pods). With **Autopilot**, Google runs the control plane across several zones of the region and adds or removes nodes automatically, so you never size a server. Autopilot also switches on **Dataplane V2**, the networking layer that enforces the NetworkPolicy in Part D.

### Option 1: Click (the console)

1. Click the menu (three lines, top left), then **Kubernetes Engine**, then **Clusters**.
2. Click **Create**. If Google asks you to enable the Kubernetes Engine API, click **Enable** and wait about a minute.
3. Under **Autopilot**, click **Configure**. Do not pick Standard: the policy test in Part D can silently fail on Standard.
4. Name: silverlake-public. Region: us-central1. If a "Navigate at a glance" tour card covers the Name box, close it with its X.
5. Leave everything else as it is and click **Create** at the bottom.
6. Wait until the cluster shows a green check (5 to 15 minutes). Read Part B while you wait.

> **Verify (console)**
>
> Kubernetes Engine > Clusters lists **silverlake-public** with a **green check**, Mode **Autopilot**, Location **us-central1**. A spinner means it is still being created; keep waiting.

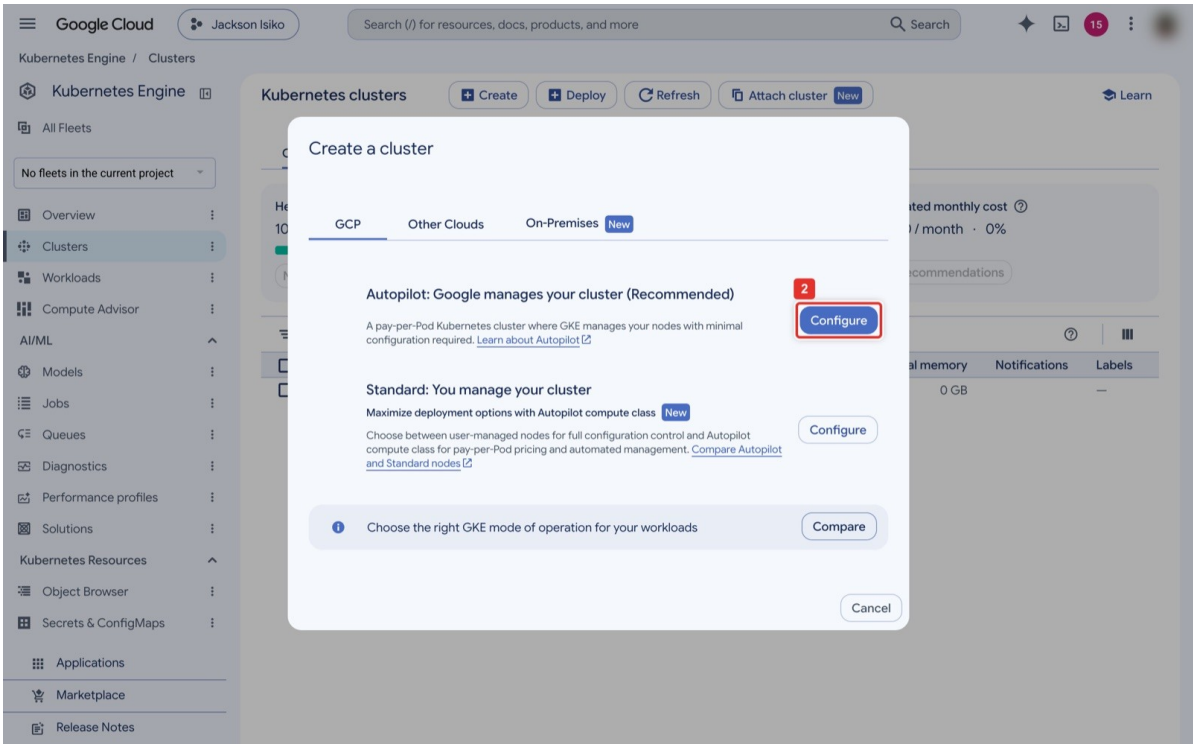



*Figure 1. Step 3: click Configure under Autopilot.*

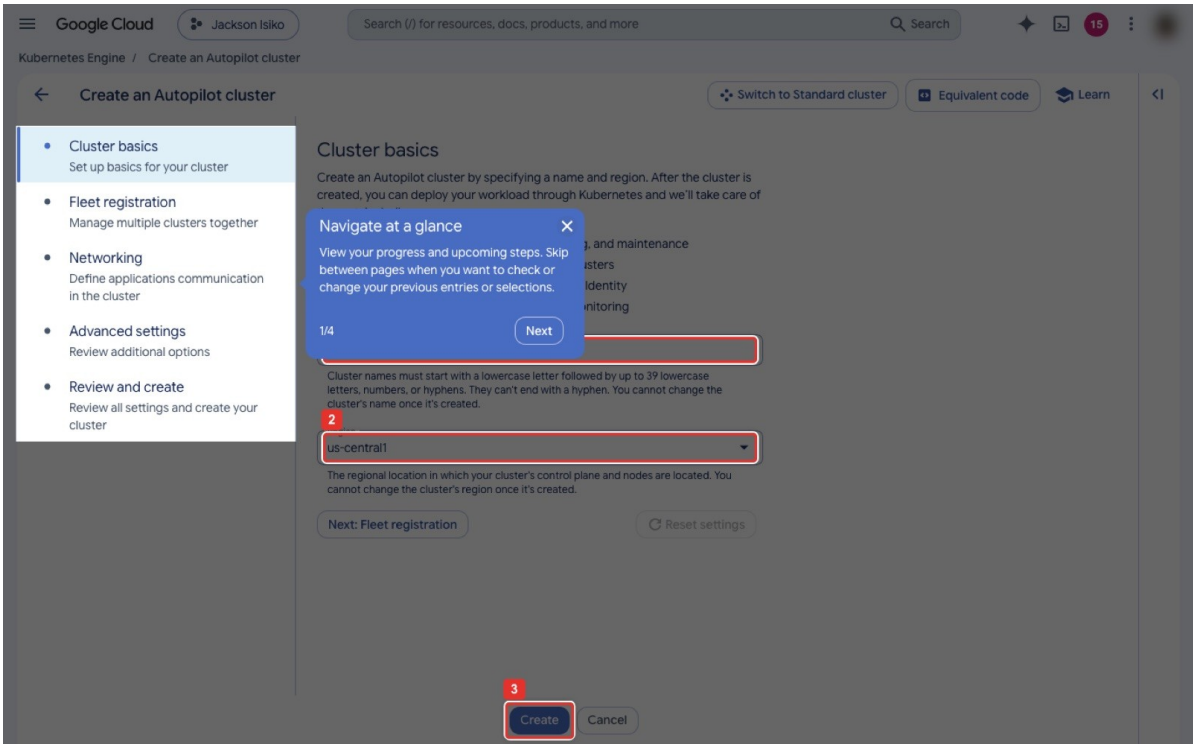



*Figure 2. Steps 4 and 5: name, region, then Create at the bottom.*

**What this step does:** the console form and the command below do the same thing: they ask GKE to build an Autopilot cluster named silverlake-public in region us-central1. Creation takes a few minutes because Google provisions and secures the control plane for you.

### Option 2: Type (Cloud Shell), same result

▸ **Run in: Cloud Shell (Google Cloud console, the >_ icon at the top right). Any folder.**

```
gcloud container clusters create-auto silverlake-public --region us-central1
```

### Then, either way: connect kubectl to the cluster

**What this step does:** get-credentials writes the cluster's address and a sign-in helper into your kubectl settings (the file ~/.kube/config), so every later kubectl and helm command goes to this cluster. clusters list confirms the cluster is RUNNING.

▸ **Run in: Cloud Shell (Google Cloud console, the >_ icon at the top right). Any folder.**

```
gcloud container clusters get-credentials silverlake-public --region us-central1
gcloud container clusters list
```

**You should see**

```
kubeconfig entry generated for silverlake-public.
NAME              LOCATION     MASTER_VERSION      STATUS
silverlake-public  us-central1  1.36.4-gke.2377000  RUNNING
```

STATUS **RUNNING** is Validation 1. Take a screenshot of the cluster list (Screenshot 1, part one).

**If not**

**does not have enough resources available**: Google has no room in that region right now. Delete the failed cluster (gcloud container clusters delete silverlake-public --region us-central1 --quiet) and create it again with --region us-east1. Use us-east1 in every later command too.

**INTERNAL error after 20+ minutes**: a temporary Google problem. Delete the cluster and create it again.

**kubectl prints HTML or "connection refused"**: you skipped get-credentials. Run it again.

# Part B. Deploy with kubectl (Deployment engineer, 10 min)

**Goal:** three copies (pods) of the web server running, and proof that Kubernetes replaces a pod that dies.

> **What this part does, and why**
>
> Kubernetes is **declarative**: you describe the result you want in a YAML file (here: three copies of NGINX 1.27) and Kubernetes works continuously to make reality match it. The **Deployment** you create makes a **ReplicaSet**, which makes the three **Pods**; the scheduler places them on nodes. If a pod dies, the ReplicaSet notices that two is less than three and starts a new one. This loop of "compare, then fix" is called **reconciliation**, and it is what lets cloud applications recover from failures without a person.

### Step 1. Get the Deployment file

**What this step does:** it copies the Deployment description (the YAML file) from the class repository into your Cloud Shell home folder and prints it so you can read it before you use it.

Download it from the class repository on GitHub (one line, nothing to retype), then look at it:

**▸ Run in: Cloud Shell (Google Cloud console, the >_ icon at the top right), in your home folder (where Cloud Shell opens; type cd ~ first if you moved). Paste the four lines one at a time; the two R= lines only build the address so it fits on the page (there is no https:// to type: -L lets curl follow GitHub to the secure address). The file is saved in your home folder; ls shows it.**

```
cd ~
R=raw.githubusercontent.com/isijackson/IT360-Fall-Labs-v2/main
R="$R/Week%203%20labs/gke-helm-networkpolicy/k8s"
curl -fsSLO "$R/nginx-deployment.yaml"
cat nginx-deployment.yaml
```

> **Verify the download (Cloud Shell, home folder)**
>
> ls -l nginx-deployment.yaml
> You should see a size of **344** bytes:
>
> -rw-r--r-- 1 <you> <you> 344 Nov 10 09:00 nginx-deployment.yaml
> A size of 0, or No such file or directory, means the download failed: run the curl line again.

Inside the file (for reading; YAML needs these exact spaces, so download it rather than copying it from a PDF):

**Inside the file (for reading only; do not paste this into Cloud Shell)**

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: loan-calc
  labels:
    app: loan-calc
spec:
  replicas: 3
  selector:
    matchLabels:
      app: loan-calc
  template:
    metadata:
      labels:
        app: loan-calc
    spec:
      containers:
      - name: nginx
        image: nginx:1.27
        ports:
        - containerPort: 80
```

`replicas: 3` is the important line: you are telling Kubernetes "always keep three of these running".

### Step 2. Apply it and wait

**What this step does:** `kubectl apply` sends the file to the cluster's API server, which stores it as the desired state. The Deployment then creates a ReplicaSet and three pods, and Autopilot adds a node if there is no room. `rollout status` waits until all three pods are ready, and `get pods` lists them.

**▸ Run in: Cloud Shell (Google Cloud console, the `>_` icon at the top right), in the same folder as `nginx-deployment.yaml` (your home folder).**

```
kubectl apply -f nginx-deployment.yaml
kubectl rollout status deployment/loan-calc
kubectl get pods
```

**You should see**

```
deployment.apps/loan-calc created
deployment "loan-calc" successfully rolled out
NAME                        READY   STATUS    RESTARTS   AGE
loan-calc-78dcf96fdc-7ntzj   1/1     Running   0          2m58s
loan-calc-78dcf96fdc-89fvv   1/1     Running   0          2m58s
loan-calc-78dcf96fdc-9pnnh   1/1     Running   0          2m58s
```

A warning that starts `autopilot-default-resources-mutator` is normal: Autopilot filled in a CPU size for you. The first rollout can take 2 to 3 minutes while Autopilot adds a machine.

**If not**

**curl prints `(23) Failed writing received data` or `(56) Failure writing output`, or `cat` says No such file**: you are not in your home folder (the prompt shows `cloudshell:/` instead of `cloudshell:~`), or a line was changed while copying. If you see `s://raw...: No such file or directory` or `curl: (3) URL using bad/illegal format`, a space crept into a line: paste the lines again one at a time. Run `cd ~` and paste the lines again exactly as printed. Still failing? Create the same Deployment with one command: `kubectl create deployment loan-calc --image=nginx:1.27 --replicas=3` (it uses the same name and the same `app=loan-calc` label).

**error converting YAML / mapping values are not allowed**: the file lost its spaces (usually copied from a PDF). Delete it (`rm nginx-deployment.yaml`) and download it with curl.

### Step 3. Break it on purpose: delete a pod

**What this step does:** it simulates a crash by deleting one pod. Kubernetes sees only two pods where you asked for three and starts a replacement within seconds, with a new name. This is the self-healing the infrastructure manager is paying for.

Copy one pod name from your own list (not from this guide), then:

▸ **Run in: Cloud Shell (Google Cloud console, the >_ icon at the top right). Any folder.**

```
kubectl delete pod <one-pod-name>
kubectl get pods
```

**You should see**

```
pod "loan-calc-78dcf96fdc-7ntzj" deleted
loan-calc-78dcf96fdc-89fvv   1/1     Running   0          3m7s
loan-calc-78dcf96fdc-9pnnh   1/1     Running   0          3m7s
loan-calc-78dcf96fdc-knb66   1/1     Running   0          8s
```

Still three pods. The new one is only seconds old. Take **Screenshot 1** (three pods Running, one of them new).

**Checkpoint: who restored the pod?**

The **ReplicaSet** behind the Deployment. It saw two pods where you asked for three and started a new one. No person was involved. Be ready to explain this to the instructor.

**Click view:** Kubernetes Engine, then Workloads, then loan-calc shows the same three pods. You can delete a pod there too: click its name, then Delete.

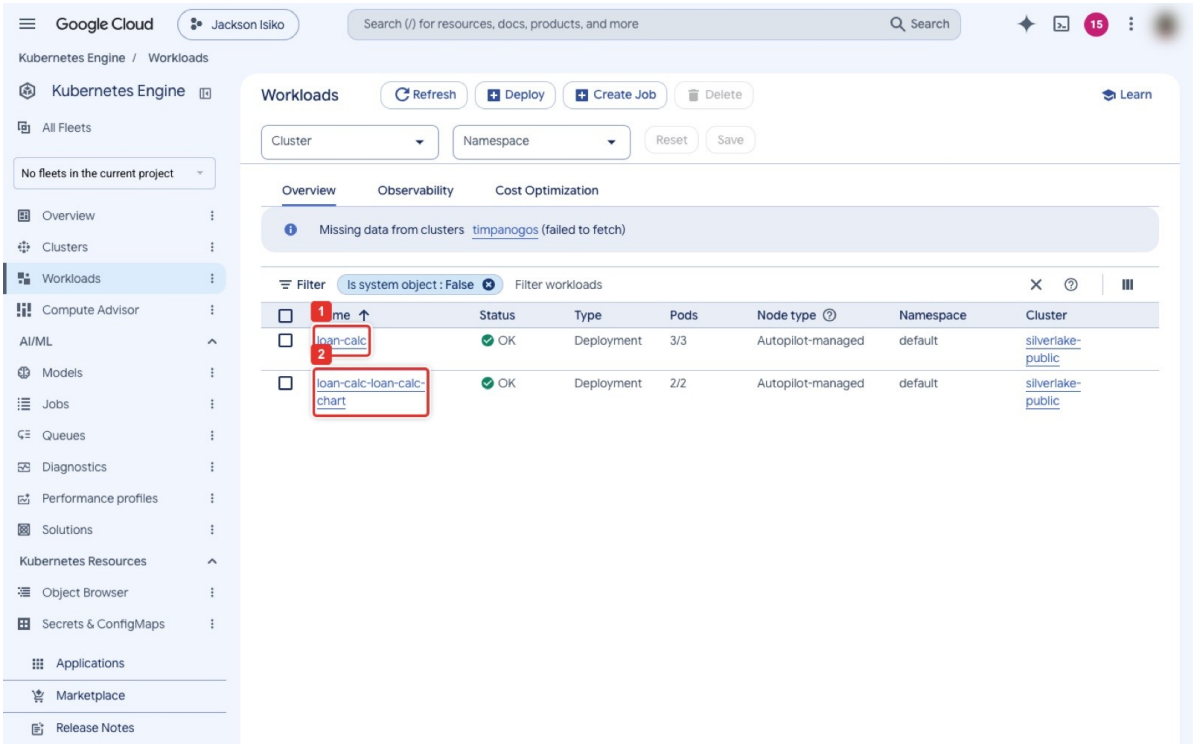



*Figure 3. Kubernetes Engine > Workloads: loan-calc (Part B) and loan-calc-loan-calc-chart (Part C).*

# Part C. Manage the app with Helm (Release engineer, 15 min)

**Goal:** the same app installed as a Helm **release**, so that every change gets a revision number and any revision can be restored with one command.

> **What this part does, and why**
>
> With plain kubectl you apply files, but nothing remembers what was there before. **Helm** adds packaging and history. A **chart** is a folder of templates (a Deployment, a Service and a few optional extras) plus a settings file, values.yaml. Installing a chart creates a **release**; every install, upgrade or rollback adds a numbered **revision**, which Helm stores inside the cluster. That gives you a one-command rollback and a record of every change, which is exactly what the auditor and the infrastructure manager asked for. The same chart can be installed unchanged on any Kubernetes cluster, on any cloud.

### Step 1. Create a chart and set two values

**What this step does:** it checks Helm is installed, creates a ready-made chart called loan-calc-chart, and edits two settings in its values.yaml: run **2** copies, using NGINX version **1.27**.

**▸ Run in: Cloud Shell (Google Cloud console, the >_ icon at the top right), in your home folder (where Cloud Shell opens; type cd ~ first if you moved). helm create makes the loan-calc-chart folder there.**

```
helm version --short
helm create loan-calc-chart
sed -i.bak 's/^replicaCount: .*/replicaCount: 2/' loan-calc-chart/values.yaml
sed -i.bak 's/^  tag: .*/  tag: "1.27"/' loan-calc-chart/values.yaml
```

helm create makes a ready-made chart for a web server. The two sed lines change it to 2 copies of NGINX 1.27. (You can open loan-calc-chart/values.yaml in the Cloud Shell Editor to see the change.)

> **Verify the two values (Cloud Shell, home folder)**
>
> ```
> grep -E '^replicaCount|^  tag' loan-calc-chart/values.yaml
> ```
> You should see:
>
> ```
> replicaCount: 2
>   tag: "1.27"
> ```
> If either line is different, run the matching sed line again.

### Step 2. Install (revision 1)

**What this step does:** helm install fills the chart's templates with your values and creates the Kubernetes objects (a Deployment and a Service named loan-calc-loan-calc-chart). This is **revision 1** of the release loan-calc. helm list shows the release.

**▸ Run in: Cloud Shell (Google Cloud console, the >_ icon at the top right), in the folder that contains loan-calc-chart (your home folder).**

```
helm install loan-calc ./loan-calc-chart
helm list
```

> **You should see**
>
> ```
> NAME      NAMESPACE  REVISION  STATUS    CHART                 APP VERSION
> loan-calc default    1         deployed  loan-calc-chart-0.1.0  1.16.0
> ```
> APP VERSION 1.16.0 is only the chart's label; the pods really run nginx 1.27 because of the tag you set.

### Step 3. Upgrade (revision 2), then roll back (revision 3)

**What this step does:** it changes the setting to 3 copies and runs helm upgrade (**revision 2**, three pods). Then helm rollback loan-calc 1 puts back exactly what revision 1 had (**revision 3**, two pods again). helm history shows all three revisions. In real teams the upgrade would be a new version of the app; the rollback is how you undo a bad release in seconds.

**▸ Run in: Cloud Shell (Google Cloud console, the >_ icon at the top right), in the folder that contains loan-calc-chart (your home folder).**

```
sed -i.bak 's/^replicaCount: .*/replicaCount: 3/' loan-calc-chart/values.yaml
helm upgrade loan-calc ./loan-calc-chart
helm rollback loan-calc 1
helm history loan-calc
```

**You should see**

```
REVISION  STATUS      CHART                 DESCRIPTION
1         superseded  loan-calc-chart-0.1.0  Install complete
2         superseded  loan-calc-chart-0.1.0  Upgrade complete
3         deployed    loan-calc-chart-0.1.0  Rollback to 1
```

Only one revision is ever **deployed**. Take **Screenshot 2:** helm list and helm history loan-calc.

**Verify the app after the rollback (Cloud Shell)**

```
kubectl get deployment loan-calc-loan-calc-chart
kubectl get deployment loan-calc-loan-calc-chart -o jsonpath='{.spec.template.spec.containers[0].image}'
```

You should see READY **2/2** (back to revision 1's two copies, not 3) and the image **nginx:1.27**:

```
NAME                      READY   UP-TO-DATE   AVAILABLE
loan-calc-loan-calc-chart   2/2     2            2
nginx:1.27
```

**Checkpoint: which revision is live?**

```
helm history loan-calc | grep deployed
```

Revision 3, which is a copy of revision 1. A rollback never erases history; it adds a new revision. That history is what an auditor reads.

**If not**

**Error: INSTALLATION FAILED: cannot re-use a name that is still in use**: you installed before. Run helm uninstall loan-calc, then Step 2 again.

**sed: can't read loan-calc-chart/values.yaml**: you are in a different folder from where you ran helm create. Run ls; if loan-calc-chart is missing, run Step 1 again.

# Part D. Lock it down with a NetworkPolicy (Security engineer, 15 min)

**Goal:** only pods labelled role=frontend may connect to the Helm release's pods, and two tests prove it: one blocked, one allowed.

> **What this part does, and why**
>
> By default every pod in a Kubernetes cluster can reach every other pod: the network is flat and open. That is the risk the auditor wrote about. A **NetworkPolicy** changes it to an allow-list: once a policy selects a group of pods for incoming (Ingress) traffic, anything it does not explicitly allow is **dropped**. Policies choose pods by **labels**, not by IP addresses, so the rule keeps working as pods are replaced and their addresses change. On GKE Autopilot, Dataplane V2 enforces the rule on every node. Applying a policy proves nothing by itself, so you then test it from both sides.

### Step 1. Get the policy and apply it

**What this step does:** it downloads the policy file, shows it, and applies it. From this moment the Helm pods accept connections only from pods labelled role=frontend.

**▸ Run in: Cloud Shell (Google Cloud console, the >_ icon at the top right), in your home folder (where Cloud Shell opens; type cd ~ first if you moved), the same folder as nginx-deployment.yaml.**

```
cd ~
R=raw.githubusercontent.com/isijackson/IT360-Fall-Labs-v2/main
R="$R/Week%203%20labs/gke-helm-networkpolicy/k8s"
curl -fsSLO "$R/network-policy.yaml"
cat network-policy.yaml
kubectl apply -f network-policy.yaml
kubectl get networkpolicy
```

> **Verify the download (Cloud Shell, home folder)**
>
> ls -l network-policy.yaml
> You should see a size of **292** bytes. Then check the policy Kubernetes stored:
>
> ```
> kubectl describe networkpolicy loan-calc-frontend-only
>  PodSelector:     app.kubernetes.io/name=loan-calc-chart
>  Allowing ingress traffic:
>    From:
>      PodSelector: role=frontend
>  Policy Types: Ingress
> ```

Inside the file:

**Inside the file (for reading only; do not paste this into Cloud Shell)**

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: loan-calc-frontend-only
spec:
  podSelector:
    matchLabels:
      app.kubernetes.io/name: loan-calc-chart
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          role: frontend
```

Read it as a sentence: "For the pods of the loan-calc chart, allow incoming traffic only from pods labelled role=frontend." Everything else is dropped.

**You should see**

```
networkpolicy.networking.k8s.io/loan-calc-frontend-only created
NAME                  POD-SELECTOR                        AGE
loan-calc-frontend-only   app.kubernetes.io/name=loan-calc-chart   0s
```

### Step 2. Save the service name in a variable

**What this step does:** it looks up the name of the Helm release's Service and keeps it in the variable $SVC. Inside the cluster that name is also a DNS name, so the test pods can reach the app by name.

**▸ Run in: Cloud Shell (Google Cloud console, the >_ icon at the top right). Any folder, but keep using the same Cloud Shell tab for Steps 3 and 4 so $SVC is remembered.**

```
SVC=$(kubectl get svc -l app.kubernetes.io/instance=loan-calc -o jsonpath='{.items[0].metadata.name}')
echo $SVC
```

**You should see**

```
loan-calc-loan-calc-chart
```

### Step 3. Test 1: a pod WITHOUT the label must be blocked

**What this step does:** it starts a temporary BusyBox pod with **no label** and asks it to fetch the app's web page, waiting at most 5 seconds. The policy drops the connection, so the request times out. --rm deletes the test pod afterwards.

**▸ Run in: Cloud Shell (Google Cloud console, the >_ icon at the top right), the same tab where you set $SVC.**

```
kubectl run probe --rm -it --image=busybox:1.36 --restart=Never -- wget -qO- --timeout=5 http://$SVC
```

**You should see (this is a PASS)**

```
wget: download timed out
pod "probe" deleted from default namespace
pod default/probe terminated (Error)
```

**Timed out = blocked = what the auditor wants.** The word Error only means wget gave up, which is the point. The command takes up to a minute because Autopilot may start a machine for the test pod first.

### Step 4. Test 2: a pod WITH role=frontend must be allowed

**What this step does:** it runs the same test from a pod that carries the label role=frontend. The policy allows it, so the NGINX welcome page comes back. One label is the only difference between the two tests, which is what makes this convincing evidence.

**▸ Run in: Cloud Shell (Google Cloud console, the >_ icon at the top right), the same tab where you set $SVC.**

```
kubectl run probe-ok --rm -it --image=busybox:1.36 --restart=Never --labels=role=frontend -- wget -qO- --timeout=5 http://$SVC
```

**You should see**

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
...
```

Take **Screenshot 3**: Test 1 and Test 2 in the same terminal window, one above the other.

Lines like `All commands and output from this session will be recorded in container logs` and `If you don't see a command prompt, try pressing enter.` are printed by GKE for every test pod. They are normal; just wait.

> **If not**
>
> **Both tests print the NGINX page**: the policy is not enforced. You probably created a Standard cluster. Autopilot enforces it; recreate the cluster as Autopilot (Part A, Option 1, step 3).
>
> **Both tests time out**: check `echo $SVC` prints `loan-calc-loan-calc-chart`, and that the `--labels=role=frontend` part is in Test 2.
>
> **pods "probe" already exists**: an earlier test is still there. `kubectl delete pod probe` and run the test again.

# Part E. Test connectivity end to end (10 min)

**Goal:** check every path traffic can take, so you know not only that the app runs, but who can reach it. Compare your results with the instructor's in the last column.

> **What this part does, and why**
>
> Integrating systems means knowing every route data can take. You test four paths inside the cluster (self-healing, name lookup, blocked and allowed pod traffic) and one from outside. The optional `kubectl expose ... --type=LoadBalancer` is the most "cloud integration" step of the week: Kubernetes asks Google Cloud to build a real external load balancer, a public IP address and the firewall rules for it, and to connect them to your pods. The same command on EKS or AKS builds an AWS or Azure load balancer.

▸ **Run every command in this table in Cloud Shell**, in the same tab where you set `$SVC` in Part D.

| Path | Command | Instructor result (Nov 8) |
|---|---|---|
| Kubernetes heals a pod | `kubectl delete pod <name>` then `kubectl get pods` | New pod Running in 8 s |
| Pod finds the service by name (DNS) | `kubectl run dns --rm -it --image=busybox:1.36 --restart=Never -- nslookup $SVC` | Name resolves to the service IP |
| Pod without the label to the app | Test 1 in Part D | `download timed out` (blocked) |
| Pod with `role=frontend` to the app | Test 2 in Part D | `Welcome to nginx!` (allowed) |
| Pod to the unprotected kubectl app | `kubectl expose deployment loan-calc --name=loan-calc-internal --port=80`, then Test 1 with `http://loan-calc-internal` | `Welcome to nginx!`: the policy only guards the Helm pods |

### Optional (Canvas): open the app from the internet

**What this step does:** `kubectl expose` creates a Service of type LoadBalancer for the Part B pods. GKE sees it and creates a Google Cloud load balancer with a public IP; `--watch` shows the EXTERNAL-IP when it arrives (press Ctrl+C to stop watching).

**▸ Run in: Cloud Shell (Google Cloud console, the >_ icon at the top right). Any folder. Then open the EXTERNAL-IP address in your laptop's web browser.**

```
kubectl expose deployment loan-calc --type=LoadBalancer --name=loan-calc-public --port=80
kubectl get service loan-calc-public --watch
```

Wait until EXTERNAL-IP changes from `<pending>` to an address (44 seconds on the instructor run), press Ctrl+C, then open `http://<EXTERNAL-IP>` in your browser. The page may refuse to load for another minute or two while Google programs the load balancer; refresh before you change anything.

> **You should see**
>
> NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S)
> loan-calc-public LoadBalancer 34.118.232.243 34.68.49.240 80:31 278/TCP
> In the browser: the **Welcome to nginx!** page.

**Discuss as a team**

Why does the public page load when the policy blocks pods? Because the LoadBalancer sends traffic to the `loan-calc` pods from Part B, and the policy only protects the Helm pods from Part C. Compare `kubectl get svc loan-calc-public -o jsonpath='{.spec.selector}'` with the policy's podSelector.

**Stuck? Ask an AI assistant the right way**

Open Gemini Cloud Assist (the spark icon at the top of the console). Paste the exact error text and the command you ran, and ask what caused it. **Never** paste keys, tokens or kubeconfig files. Check any command it suggests against this guide before you run it.

# Part F. Submit, then clean up (10 min)

## What to submit in Canvas 3.2

- ☐ **Screenshot 1:** gcloud container clusters list showing RUNNING, and kubectl get pods with three pods Running after you deleted one.
- ☐ **Screenshot 2:** helm list and helm history loan-calc (revision 3, Rollback to 1).
- ☐ **Screenshot 3:** Test 1 (timed out) and Test 2 (Welcome to nginx!) in one terminal.
- ☐ The NetworkPolicy YAML (copy it from Part D).
- ☐ A 2 to 3 sentence summary that names who did what (one sentence per role is fine).

## Clean up the same day

An Autopilot cluster costs about 73 dollars a month even when idle. Run these in order once your screenshots are saved:

**What this step does:** it removes everything in the reverse order you built it. The LoadBalancer Service goes first so Google also deletes the load balancer and public IP it created; then the Helm release, the policy and the Deployment; finally the cluster itself, which is what bills by the hour. clusters list confirms nothing is left.

**▸ Run in: Cloud Shell (Google Cloud console, the >_ icon at the top right), in your home folder (where Cloud Shell opens; type cd ~ first if you moved), where the two YAML files are.**

```
kubectl delete service loan-calc-public loan-calc-internal --ignore-not-found
helm uninstall loan-calc
kubectl delete -f network-policy.yaml -f nginx-deployment.yaml
gcloud container clusters delete silverlake-public --region us-central1 --quiet
gcloud container clusters list
```

**You should see**

The last command prints nothing: no clusters left. Then end the Vocareum lab in Canvas.

Delete the LoadBalancer service **before** the cluster, so Google also removes the load balancer it created for you.

## Troubleshooting at a glance

| You see | Do this |
|---|---|
| does not have enough resources available | Recreate the cluster with --region us-east1 and use that region everywhere. |
| Pods or test pods stay Pending | Autopilot is adding a machine. Wait up to 90 seconds; kubectl get events shows progress. |
| Both tests succeed | Not Autopilot, so the policy is not enforced. Recreate as Autopilot. |
| helm install says the name is in use | helm uninstall loan-calc, then install again. |
| kubectl prints HTML or times out | Run gcloud container clusters get-credentials silverlake-public --region us-central1 again. |
| Cloud Shell disconnected and $SVC is empty | Run Part D Step 2 again; variables do not survive a reconnect. |

| You see | Do this |
|---|---|
| EXTERNAL-IP stays <pending> over 3 minutes | kubectl describe service loan-calc-public and read the Events at the bottom. |

## Words to know

| Word | Meaning in this lab |
|---|---|
| Cluster | The set of machines Kubernetes runs your app on. Autopilot manages the machines for you. |
| Pod | One running copy of the app (here: one NGINX web server). |
| Deployment | The instruction "keep N pods of this running". It replaces pods that die. |
| Service | A stable name and address in front of a group of pods. |
| Helm release | An installed app with a numbered history, so you can roll back. |
| NetworkPolicy | A firewall rule between pods, chosen by labels instead of IP addresses. |
| Label | A tag on a pod, such as role=frontend. Policies and Services select pods by label. |
| ReplicaSet | The controller a Deployment creates; it keeps the exact number of pods running. |
| Control plane | The cluster's brain: the API server kubectl talks to, the scheduler and the state database. Google runs it on Autopilot. |
| Chart / values.yaml | A Helm package of templates, and the settings file that fills them in. |
| Revision | One numbered version of a Helm release; rollback makes a new revision that copies an old one. |
| Dataplane V2 | GKE's eBPF-based networking that enforces NetworkPolicy on every node. |
| LoadBalancer Service | A Service that makes the cloud provider create an external load balancer and public IP for your pods. |